

FSCT8561-Lab 1

Part 7

```
561-Labs-Alvin-Sin/Lab1$ python3 client.py
Enter your username: Alvin
Server: OK|Hello Alvin
Enter message or type EXIT to leave: Hello
Server: OK|Message received from Alvin
Enter message or type EXIT to leave: how are you
Server: OK|Message received from Alvin
Enter message or type EXIT to leave: EXIT
Server: OK|Goodbye
Disconnected
```

```
561-Labs-Alvin-Sin/Lab1$ python3 server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 54880)
Received: HELLO|Alvin
Username: Alvin
Received: MSG|Hello
Alvin says: Hello
Received: MSG|how are you
Alvin says: how are you
Received: EXIT|
Server closed
```

Part 8

```
561-Labs-Alvin-Sin/Lab1$ python3 client.py
Enter your username:
Server: ERROR|Username required
Enter message or type EXIT to leave: test
Server: ERROR|Unknown command
Enter message or type EXIT to leave: ^C
Traceback (most recent call last):
  File "/media/costhm/Hard-Disk/FSCT8561-Labs-Alvin-Sin/Lab1/client.py", line 27, in <module>
    message = input(
              ^^^^^^
KeyboardInterrupt
```

```

● 561-Labs-Alvin-Sin/Lab1$ python3 server.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 45318)
Received: HELLO|
Received: BADCOMMAND|test
Client disconnected unexpectedly
Server closed

```

In the above screenshots, the demonstration for the robustness of the code are as following: Empty Message, Bad command, and client disconnected unexpectedly. As shown in the screenshot, the error is handled properly even with temporary tampering with client-side code.

Part 10

1. In this lab, Error is not handled properly on client side, will prompt for initial message. As the TCP connection closes, on cliend side a call to `recv()` will return 0 bytes when connection is closed, or eventually raise a socket error if the connection is closed.
2. The server could use asynchronous I/O like `asyncio` in python library, which can handle many connections without creating a thread for each client
3. TCP is a byte stream, not a message-based protocol, it odes not make sure a complete application message arrives in one call. `recv(1024)` means giving 1024 bytes that are currently available, not giving a complete message
4. TCP provides reliability in terms of delivery, ordering of packet, error detection and retransmission, but TCP does not provide application-level security. It does not provide authentication, encryption, authorization, protection against command injection or input validation by itself. It can be reliable but an attacker can exploit TCP's foundation.
5. Command injection, malformed commands, unauthorized commands, session abuse with accounts, brute force attempts, username enumeration. Adding more functionality means it needs to be validated and secured
6. A malicious client can send unexpected or excessive input, sending invalid commands, attempt to access another user's data or exploit the weaknesses in how commands are parsed.
7. Implement a login system with its own separate database, storing password hashes using a strong hashing algorithm/libraries like `bcrypt`. Client submits credentials over an encrypted conections and server verifies the password against the stored hash. Then creating a session token for the user. Also implementing correct access control for authorization limitations for commands that is associated with that authenticated user.
8. For TCP application, adding TLS would be the first thing when it comes to providing encryption for an application, it also helps with authenticating the server with certificates.
9. Command format: checking whether the message contains "|" separator, if not, return an error
 Username: "HELLO" command checks that the username is not empty
 Ordering: requires the client to send "HELLO" before allowing it to send MSG
 Message content: check if the message is empty or not
 Message length: limiting message length to 200 characters

Command validation: if the command is not “HELLO” “MSG” or “EXIT”, returns an error of unknown command

Connection Handling: Checks for an empty `recv()` result and handles Error when the client disconnects unexpectedly

10. Encryption: with TLS

Authentication: strong password hashing, and secure authentication

Authorization: Permissions for different operations based on user

Input: Strict validation and size limit, with sanitation

Brute force protection: rate limiting or login attempts limits

DoS: Connection and resource limits

Sessions: Secure session tokens and TTL

Logging: Security event logging and monitoring

Error Handling: Proper error handling, prevent revealing too much information

Dependencies: keep libraries and operating system updated

Secrets: don't put secret keys, APIs on github, don't hard code keys

Network Security: Firewall, Exposed ports

Part 11

```
CT-8561/labs/Lab0$ python3 server.py
===== Running on http://0.0.0.0:5000 =====
(Press CTRL+C to quit)
Client connected: LC6tWXqr3pF2Se0dAAAB
Received from LC6tWXqr3pF2Se0dAAAB: Hello from the client!
Client disconnected: LC6tWXqr3pF2Se0dAAAB
□
```

```
CT-8561/labs/Lab0$ python3 client.py
websocket-client package not installed, only polling transport is available
Server says: Welcome!
Connected to server
Server says: Server received: Hello from the client!
^CDisconnected from server
```

```
import socketio
from aiohttp import web

# Create the Socket.IO server
sio = socketio.AsyncServer(
    async_mode="aiohttp",
    cors_allowed_origins="*"
)
```

```

# App start
app = web.Application()
sio.attach(app)

# Send message to client when connected
@sio.event
async def connect(sid, environ):
    print(f"Client connected: {sid}")
    await sio.emit("message", "Welcome!", to=sid)

# Logs when client disconnects
@sio.event
async def disconnect(sid):
    print(f"Client disconnected: {sid}")

# Handles message from client
@sio.event
async def message(sid, data):
    print(f"Received from {sid}: {data}")

    # Send a response back to that client
    await sio.emit(
        "message",
        f"Server received: {data}",
        to=sid
    )

if __name__ == "__main__":
    web.run_app(app, host="0.0.0.0", port=5000)

```

```

# Client
import socketio
sio = socketio.Client()

@sio.event
def connect():
    print("Connected to server")

    # Send a message to the server
    sio.emit("message", "Hello from the client!")

@sio.event

```

```
def disconnect():
    print("Disconnected from server")

@sio.event
def message(data):
    print("Server says:", data)

sio.connect("http://localhost:5000")
sio.wait()
```

Discussion

1. Which low-level operations from this lab are hidden by [Socket.IO](#)?

[Socket.io](#) hides the socket being created, connection using `connect()`, binding the server to an IP address and port, listening for connections, accepting clients, sending and receiving data, detect client disconnections, manage TCP connection state, decides how messages are separated or framed.

Using [Socket.IO](#), all of these operations are handled by the library, with it handles much of the underlying networking communication instead of manually managing the connection and data transfer.

Challenge

```
python3 server.py
Server is waiting for connections...
Connected by: ('127.0.0.1', 60118)
Active threads: 1
Received: HELLO|Alice
Username: Alice
Connected by: ('127.0.0.1', 58384)
Active threads: 2
Received: HELLO|Bob
Username: Bob
Received: MSG|Hello everyone!
Alice: Hello everyone!
Received: MSG|Hi Alice!
Bob: Hi Alice!
```

```
python3 client.py
Enter your username: Alice
Server: OK|Hello Alice
> Hello everyone!
>
Server: OK|Message sent

Server: Bob: Hi Alice1
```

```
python3 client.py
Enter your username: Bob
Server: OK|Hello Bob
>
Server: Alice: Hello everyone!
Hi Alice1
>
Server: OK|Message sent
```

1. Why could the original Lab 1 server handle only one client at a time
 - a. Because it called `accept()` once and stayed in a loop handling that client, it could not accept another connection until the first client disconnected
2. What problem does threading solve?
 - a. Threading allows multiple client to be handled by a separate thread so multiple clients can communicate with the server continuously
3. What happens to the other clients when one client disconnects?
 - a. The disconnected client's thread ends, but the other clients and their threads continue communicating with the server normally
4. What additional security risks arise when multiple clients share the same server?
 - a. Risks like unauthorized access, DoS, malicious messages and attacks that exploit shared server resources might occur
5. What could happen if two clients try to use the same username?
 - a. They could be confused with each other, making it difficult to identify who sent a message, server should handle duplicated usernames
6. How would you prevent one user from impersonating another user?
 - a. Using a proper authentication protocol and associating the authenticated user's identity with their socket/session. Don't allow the client to claim any username

Security analysis

This lab demonstrated how a basic client-server chat application works using raw TCP sockets and how the design changes when supporting multiple clients. From a security perspective, the lab showed that reliable network communication does not automatically make an application secure.

The original server used TCP sockets to accept a client, receive data with `recv(1024)`, process commands, and send responses. Basic input validation was implemented by checking the command format, requiring a username for `HELLO`, requiring `HELLO` before allowing `MSG`, rejecting empty messages, and limiting messages to 200 characters. Unknown commands were also rejected. These checks help prevent simple malformed input from being processed incorrectly. However, the username was only checked to ensure it was not empty, so a client could still choose another user's name.

The multi-client challenge introduced threading so that each connected client could be handled independently. The server maintains a mapping between client sockets and usernames and forwards messages to other connected clients. This introduces additional security concerns because multiple users now share the same server resources. A malicious client could send large amounts of data, repeatedly connect, or send malformed commands in an attempt to consume server resources or cause unexpected behavior. Concurrent access to shared data such as the client list can also create synchronization issues if the application becomes more complex.

The lab also demonstrated the difference between raw TCP sockets and higher-level libraries such as `Socket.IO`. `Socket.IO` hides many low-level networking operations and provides events such as `connect`, `message`, and `disconnect`. While this makes development easier, it also abstracts away some of the underlying communication details that must be understood when working directly with TCP.

The current implementation does not provide authentication or encryption. A client can potentially impersonate another user by choosing the same username, and data transmitted over a plain TCP connection can be observed by someone able to monitor the network. A real deployment should use authentication, authorization, TLS encryption, stronger username validation, rate limiting, connection limits, timeouts, and proper message framing. It should also protect shared server state and log security-relevant events.

Overall, the lab demonstrates that network reliability, input validation, authentication, and encryption are separate security concerns. A functional chat application is only the starting point; additional security controls are required before exposing it to an untrusted network.